

// Jivops

Guía completa — 30 lecciones

Arquitectura · Reglas y detección · Casos de uso · Integración y
respuesta · Operación a escala · Madurez y compliance

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Falco y detección de amenazas en runtime
- Lección 02 — Arquitectura de Falco — kernel module, eBPF, userspace
- Lección 03 — Instalación de Falco (Helm, DaemonSet)
- Lección 04 — Falco Rules — sintaxis y estructura
- Lección 05 — Falcosidekick — enrutamiento de eventos

Módulo 2: Reglas y detección

- Lección 06 — Reglas por defecto de Falco — qué cubren
- Lección 07 — Escritura de reglas personalizadas
- Lección 08 — Macros y listas reutilizables
- Lección 09 — Condiciones y campos de eventos del kernel
- Lección 10 — Priority levels y su significado operativo

Módulo 3: Casos de uso de detección

- Lección 11 — Detección de shells interactivas en contenedores
- Lección 12 — Detección de escritura en directorios sensibles
- Lección 13 — Detección de escalación de privilegios
- Lección 14 — Detección de exfiltración de datos
- Lección 15 — Detección de conexiones de red sospechosas

Módulo 4: Integración y respuesta

- Lección 16 — Falcosidekick hacia Slack/Teams
- Lección 17 — Integración con SIEM (Elasticsearch, Splunk)
- Lección 18 — Respuesta automática con Falco Talon
- Lección 19 — Integración con Kubernetes (kubectl exec detection)
- Lección 20 — Correlación con audit logs de Kubernetes

Módulo 5: Operación a escala

- Lección 21 — Tuning de reglas — reducción de falsos positivos
- Lección 22 — Rendimiento — overhead de Falco en producción
- Lección 23 — Gestión de reglas a escala (múltiples clusters)
- Lección 24 — Actualización de Falco sin downtime
- Lección 25 — Troubleshooting de reglas que no disparan

Módulo 6: Madurez y compliance

Lección 26 — Falco y compliance (CIS Benchmarks, PCI-DSS)

Lección 27 — Auditoría de eventos de Falco

Lección 28 — Gestión de excepciones y whitelisting

Lección 29 — Falco en entornos multi-tenant

Lección 30 — Proyecto final: sistema de detección de amenazas runtime completo con Falco

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Falco y detección de amenazas en runtime

Falco detecta comportamiento anómalo mientras las aplicaciones se ejecutan, complementando (no reemplazando) el escaneo de imágenes antes del despliegue.

Lección 02 — Arquitectura de Falco — kernel module, eBPF, userspace

Falco observa syscalls directamente en el kernel, dando visibilidad real independiente de si la aplicación registra ese comportamiento en sus propios logs.

Lección 03 — Instalación de Falco (Helm, DaemonSet)

Se despliega como DaemonSet porque el kernel es específico de cada nodo — Falco necesita ejecutarse localmente en cada uno para observarlo.

```
helm install falco falcosecurity/falco
```

Lección 04 — Falco Rules — sintaxis y estructura

Separar condition (lógica) de output (mensaje) permite entender qué se detecta en lenguaje natural, sin conocer a fondo la sintaxis técnica interna.

Lección 05 — Falcosidekick — enrutamiento de eventos

Al estar desacoplado del motor de detección, se puede añadir un nuevo destino de alertas sin tocar ni reescribir ninguna regla existente.

Módulo 2: Reglas y detección

Lección 06 — Reglas por defecto de Falco — qué cubren

Las reglas por defecto ya cubren patrones de ataque comunes validados por la comunidad, un punto de partida sólido antes de personalizar.

Lección 07 — Escritura de reglas personalizadas

Combinar `proc.name` con `container.id` asegura que la alerta solo dispare dentro de contenedores, distinguiendo ese contexto del proceso normal en el host.

```
proc.name = "bash"
```

Lección 08 — Macros y listas reutilizables

Reutilizar una macro común en varias reglas evita duplicar la misma lógica, permitiendo actualizar la condición en un solo lugar si cambia.

```
evt.type = execve
```

Lección 09 — Condiciones y campos de eventos del kernel

Combinar el tipo de evento (`open_write`) con `fd.name` filtra específicamente escrituras en un directorio sensible, no cualquier escritura de archivo.

```
fd.name startswith /etc
```

Lección 10 — Priority levels y su significado operativo

Alertar solo en priority CRITICAL/EMERGENCY enfoca la atención inmediata del equipo en lo urgente, sin saturar con eventos de menor severidad.

Módulo 3: Casos de uso de detección

Lección 11 — Detección de shells interactivas en contenedores

Una shell interactiva inesperada en un contenedor de un solo proceso puede indicar acceso no autorizado, aunque también ocurre en debugging legítimo.

Lección 12 — Detección de escritura en directorios sensibles

Un contenedor inmutable bien diseñado no debería necesitar escribir en /etc durante su ejecución normal, ya que su configuración vive en el build.

Lección 13 — Detección de escalación de privilegios

Un cambio inesperado de UID hacia mayores privilegios durante la ejecución sugiere que algo anómalo, posiblemente un exploit, está ocurriendo.

Lección 14 — Detección de exfiltración de datos

Combinar lectura de archivo sensible con conexión de red saliente en secuencia forma un patrón más indicativo que cada señal por separado.

Lección 15 — Detección de conexiones de red sospechosas

Una allowlist de IPs legítimas suele ser más manejable que una blocklist de amenazas conocidas, que crece constantemente y nunca está completa.

```
outbound and not fd.sip in (allowed_ips)
```

Módulo 4: Integración y respuesta

Lección 16 — Falcosidekick hacia Slack/Teams

Configurar el destino de notificación en Falcosidekick centraliza el cambio: basta actualizar la configuración una vez para cambiar el canal.

Lección 17 — Integración con SIEM (Elasticsearch, Splunk)

Correlacionar eventos de Falco con otras fuentes en un SIEM da el contexto completo necesario para entender el alcance real de un incidente.

Lección 18 — Respuesta automática con Falco Talon

Aislar automáticamente un Pod comprometido contiene el daño mientras el equipo investiga, sin esperar una reacción manual que podría ser tardía.

Lección 19 — Integración con Kubernetes (kubectl exec detection)

Falco ve el comportamiento del proceso; los audit logs de Kubernetes ven qué usuario hizo la llamada — juntos dan el contexto completo de 'quién hizo qué'.

Lección 20 — Correlación con audit logs de Kubernetes

Falco opera a nivel de kernel del contenedor; la identidad del usuario que llamó a la API vive en una capa distinta que solo el audit log registra.

Módulo 5: Operación a escala

Lección 21 — Tuning de reglas — reducción de falsos positivos

Una excepción acotada silencia solo el patrón ya identificado como legítimo, manteniendo el resto de la protección de la regla intacta.

Lección 22 — Rendimiento — overhead de Falco en producción

Menos reglas activas y más enfocadas reduce el trabajo de evaluación por cada syscall observada, disminuyendo el overhead general.

Lección 23 — Gestión de reglas a escala (múltiples clusters)

Versionar reglas en Git con GitOps permite auditar y revertir un cambio problemático propagándolo de forma consistente a todos los clusters.

Lección 24 — Actualización de Falco sin downtime

Un rolling update mantiene cobertura parcial durante la transición, evitando una ventana sin ninguna detección activa en todo el cluster.

```
kubectl rollout status daemonset falco -n falco
```

Lección 25 — Troubleshooting de reglas que no disparan

Un error de sintaxis podría hacer que Falco rechace una regla silenciosamente, sin ningún síntoma visible desde la aplicación monitoreada.

```
kubectl logs -n falco falco-xxx
```


Módulo 6: Madurez y compliance

Lección 26 — Falco y compliance (CIS Benchmarks, PCI-DSS)

Un auditor necesita evidencia de que el control realmente funciona en la práctica, no solo confirmar que la herramienta está técnicamente instalada.

Lección 27 — Auditoría de eventos de Falco

Sin retención de eventos, la evidencia de un ataque descubierto semanas después se habría perdido, imposibilitando reconstruir la línea de tiempo.

Lección 28 — Gestión de excepciones y whitelisting

Una excepción muy amplia (como 'cualquier bash') crea un punto ciego en todo el cluster, mientras una acotada limita el riesgo solo al caso previsto.

Lección 29 — Falco en entornos multi-tenant

Exponer las alertas de un tenant a otro rompería la expectativa básica de aislamiento y confianza en un cluster compartido.

Lección 30 — Proyecto final: sistema de detección de amenazas runtime completo con Falco

Se integra todo lo anterior: reglas por defecto y personalizadas ajustadas, Falcosidekick hacia SIEM/Slack, respuesta automática con Talon, retención de eventos y tuning de falsos positivos — el nivel esperado de un sistema de detección runtime con Falco listo para producción real.